

API I TESTY JEDNOSTKOWE

SPIS TREŚCI

Spis treści	1
Cel zajęć.....	1
Rozpoczęcie	1
Uwaga	1
Odczyt wejścia w kontrolerze.....	2
Wykorzystanie serwisu.....	3
Format CSV	5
Wykorzystanie TWIG	8
Fahrenheit	11
Test jednostkowy.....	12
DataProvider	15
Commit projektu do GIT.....	16
Podsumowanie.....	17

CEL ZAJĘĆ

Celem głównym zajęć jest zdobycie następujących umiejętności:

- wykorzystanie reużywalnej logiki biznesowej z serwisów do tworzenia API;
- testowanie jednostkowe.

ROZPOCZĘCIE

Rozpoczęcie zajęć. Powtórzenie możliwości tworzenia API w Symfony. Omówienie testów jednostkowych, integracyjnych i funkcjonalnych.

Wejściówka?

UWAGA

Ten dokument aktywnie wykorzystuje niestandardowe właściwości. Podobnie jak w LAB A wejdź do **Plik** -> **Informacje** -> **Właściwości** -> **Właściwości zaawansowane** -> **Niestandardowe** i zaktualizuj pola. Następnie uruchom ten dokument ponownie lub **Ctrl+A** -> **F9**.

ODCZYT WEJŚCIA W KONTROLERZE

Wykorzystaj komendę `make:controller` do stworzenia kontrolera `WeatherApiController`:

```
php .\bin\console make:controller --no-template

Choose a name for your controller class (e.g. AgreeablePuppyController):
> WeatherApiController

created: src/Controller/WeatherApiController.php

Success!

Next: Open your new controller class and add some pages!
```

Zastosowanie flagi `--no-template` skutkuje wygenerowaniem kontrolera, który zwraca JSON zamiast renderowania szablonu:

```
class WeatherApiController extends AbstractController
{
    #[Route('/weather/api', name: 'app_weather_api')]
    public function index(): JsonResponse
    {
        return $this->json([
            'message' => 'Welcome to your new controller!',
            'path' => 'src/Controller/WeatherApiController.php',
        ]);
    }
}
```

Zmień ścieżkę routingu na „/api/v1/weather”, metoda pozostaje GET.

Wykorzystaj atrybuty `MapQueryParameter` do zmapowania parametrów `country` i `city` do ustawienia lokalnych zmiennych `$country` i `$city`. Więcej informacji: <https://symfony.com/blog/new-in-symfony-6-3-query-parameters-mapper>.

Na ten moment działanie kontrolera ogranicz do wyświetlenia w JSONie otrzymanych parametrów wejściowych:

```
return $this->json([
    'city' => $city,
    'country' => $country,
]);
```

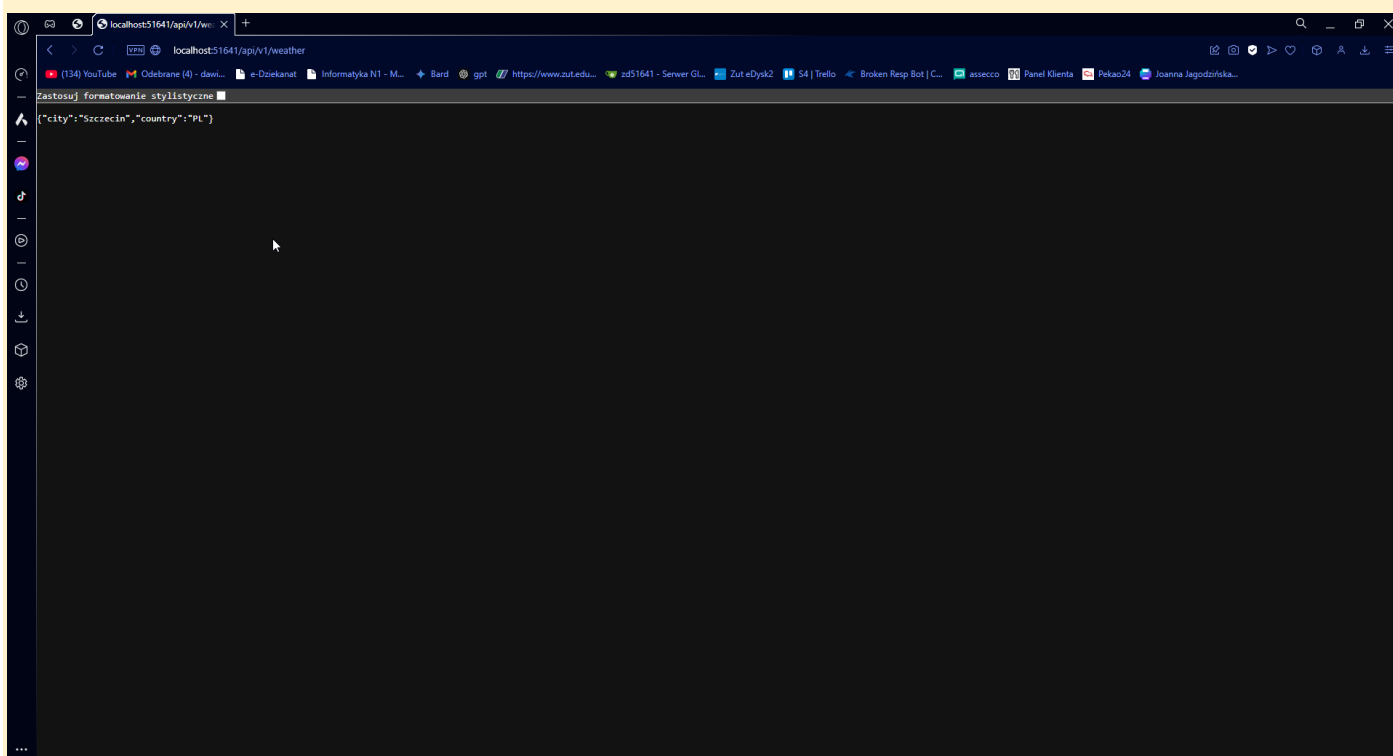
Wstaw zrzut ekranu kodu kontrolera na tym etapie:

```

class WeatherApiController extends AbstractController
{
    #[Route('/api/v1/weather', name: 'app_weather_api', methods: ['GET'])]
    public function index(
        #[MapQueryParameter('country')] string $country,
        #[MapQueryParameter('city')] string $city,
    ): Response
    {
        return $this->json([
            'city' => $city,
            'country' => $country,
        ]);
    }
}

```

Wstaw zrzut ekranu otrzymanego z kontrolera JSONa:



Punkty:

0

1

WYKORZYSTANIE SERWISU

Podłącz do akcji kontrolera serwis `WeatherUtil`. Wykorzystaj go do pobrania prognozy pogody dla zadanej miejscowości, a następnie uzupełnij JSON wynikowy o pozycję `'measurements'` – tablicę wyników.

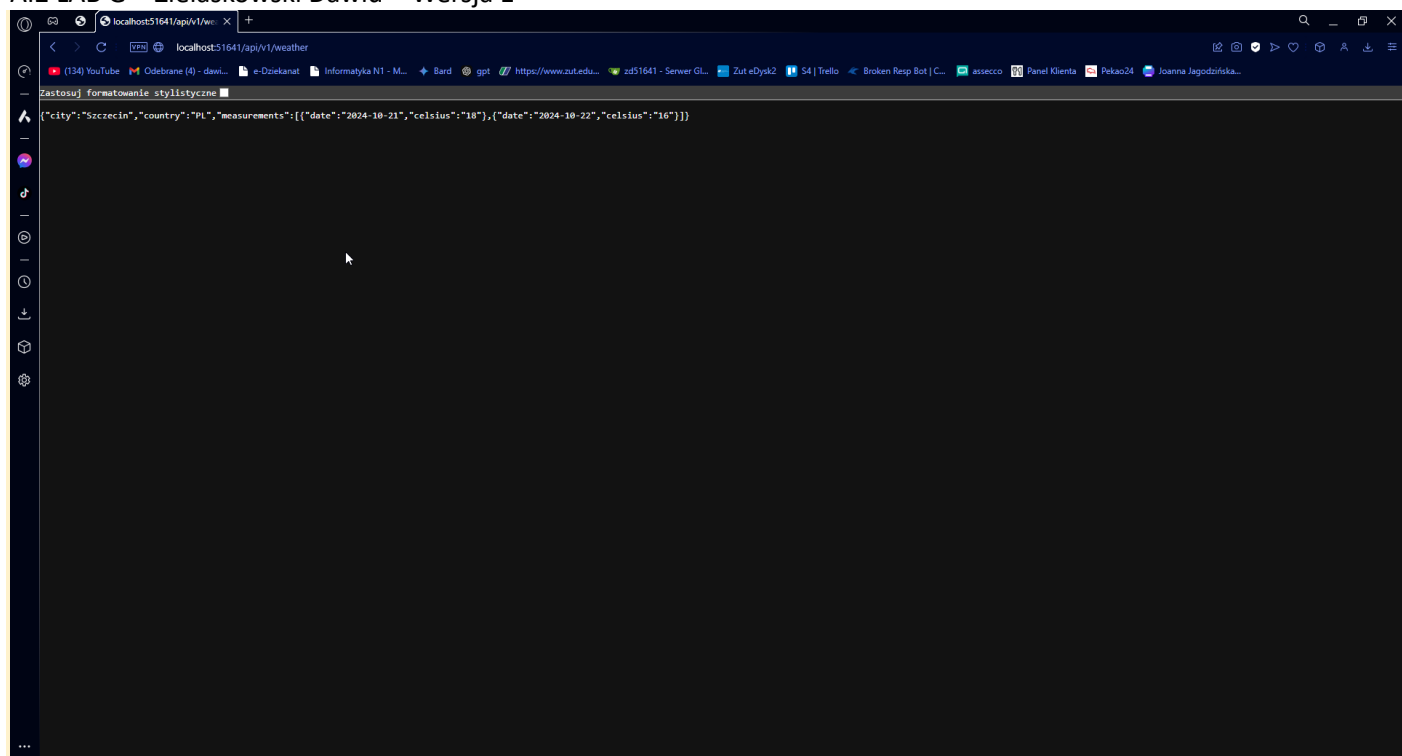
Przydatny może się okazać kod z wykorzystaniem `array_map`:

```
'measurements' => array_map(fn(Measurement $m) => [
    'date' => $m->getDate()->format('Y-m-d'),
    'celsius' => $m->getCelsius(),
], $measurements),
```

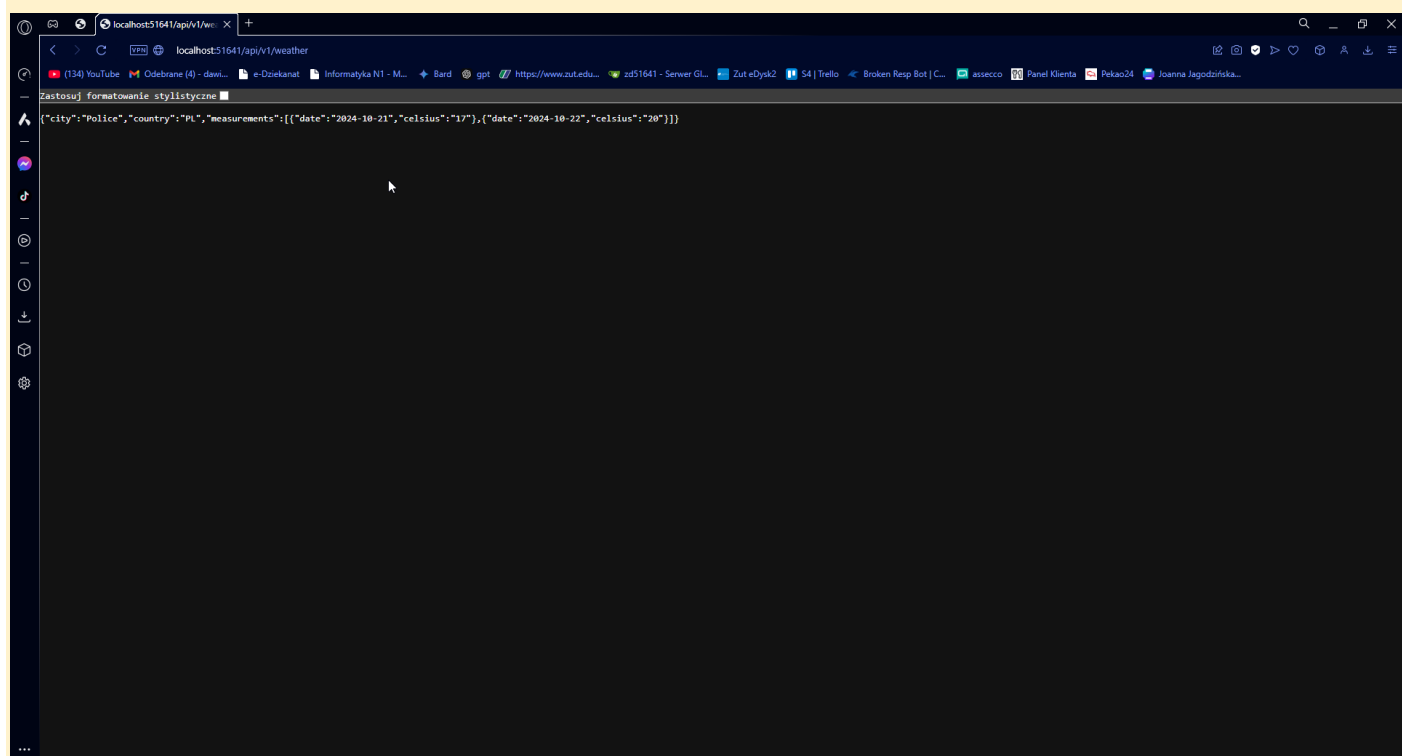
Wstaw zrzut ekranu kodu kontrolera na tym etapie:

```
class WeatherApiController extends AbstractController
{
    #[Route('/api/v1/weather', name: 'app_weather_api', methods: ['GET'])]
    public function index(
        WeatherUtil $util,
        #[MapQueryParameter('country')] string $country,
        #[MapQueryParameter('city')] string $city,
    ): Response
    {
        $measurements = $util->getWeatherForCountryAndCity($country, $city);
        return $this->json([
            'city' => $city,
            'country' => $country,
            'measurements' => array_map(fn(Measurement $m) => [
                'date' => $m->getDate()->format('Y-m-d'),
                'celsius' => $m->getCelsius(),
            ], $measurements),
        ]);
    }
}
```

Wstaw zrzuty ekranu otrzymanego z kontrolera JSONa dla dwóch miejscowości:



```
localhost:51641/api/v1/weather
Zastosuj formatowanie stylistyczne
{"city":"Szczecin","country":"PL","measurements":[{"date":"2024-10-21","celsius":18},{"date":"2024-10-22","celsius":16}]}
```



```
localhost:51641/api/v1/weather
Zastosuj formatowanie stylistyczne
{"city":"Police","country":"PL","measurements":[{"date":"2024-10-21","celsius":17},{"date":"2024-10-22","celsius":20}]}
```

Punkty:	0	1
---------	---	---

FORMAT CSV

Uzupełnij przyjmowane przez akcję kontrolera parametry o parametr format. Dopuszczalne wartości to json i csv. Dla json działanie kontrolera zostaje jak poprzednio. Dla csv zwrócony powinien zostać wynik w postaci rozdzielanej przecinkami, o kolumnach:

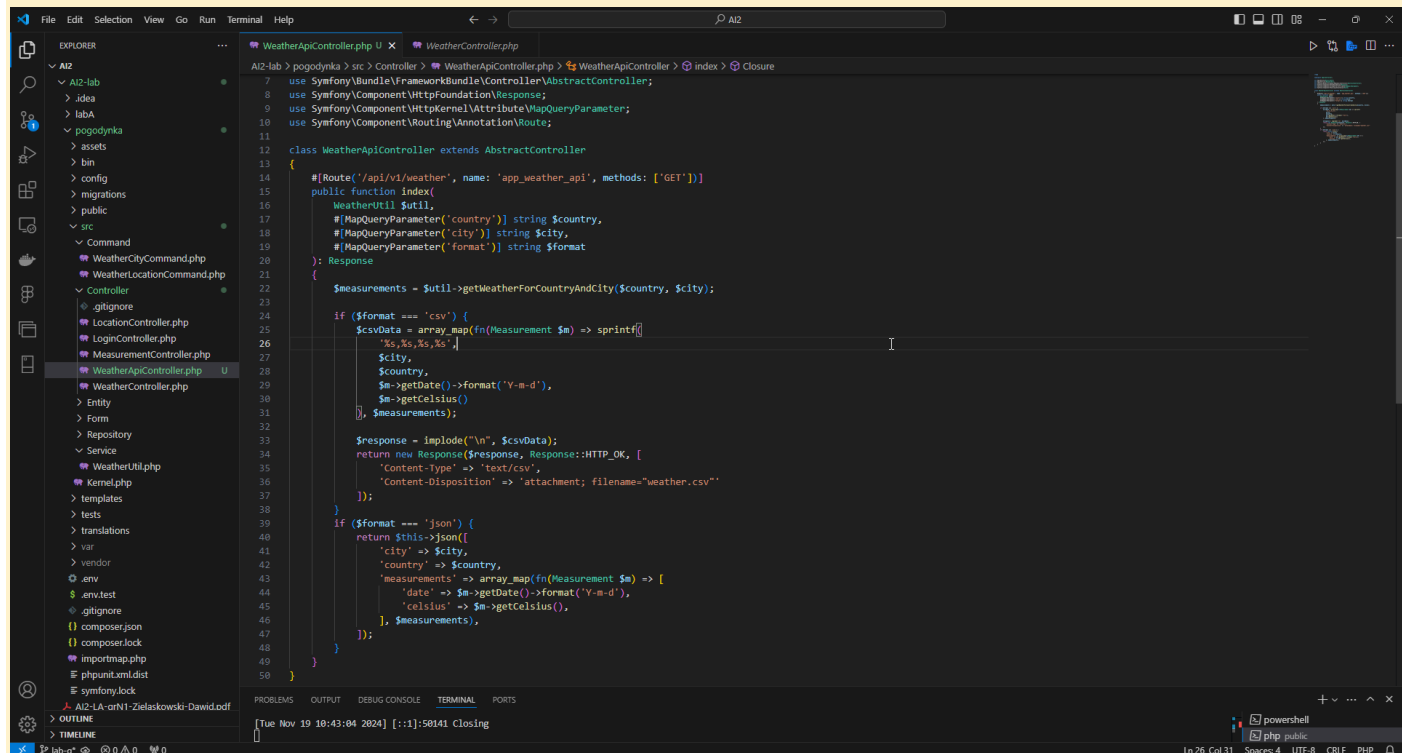
A12 LAB G – Zielaskowski Dawid – Wersja 1

- city
- country
- date
- celsius

Zwróć uwagę, że city i country podawane będą redundantnie w każdej linii.

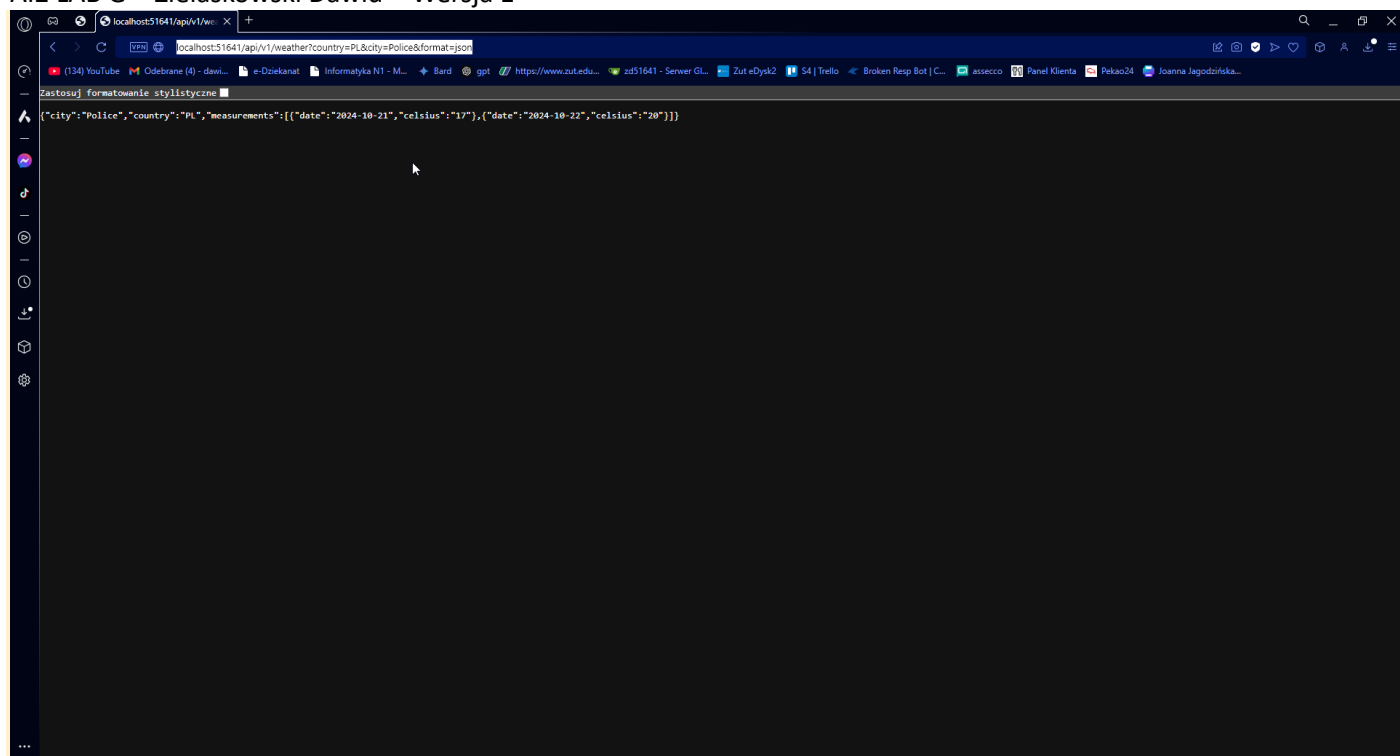
Wykorzystaj funkcję `sprintf()` albo `implode()`.

Wstaw zrzut ekranu kodu kontrolera na tym etapie:

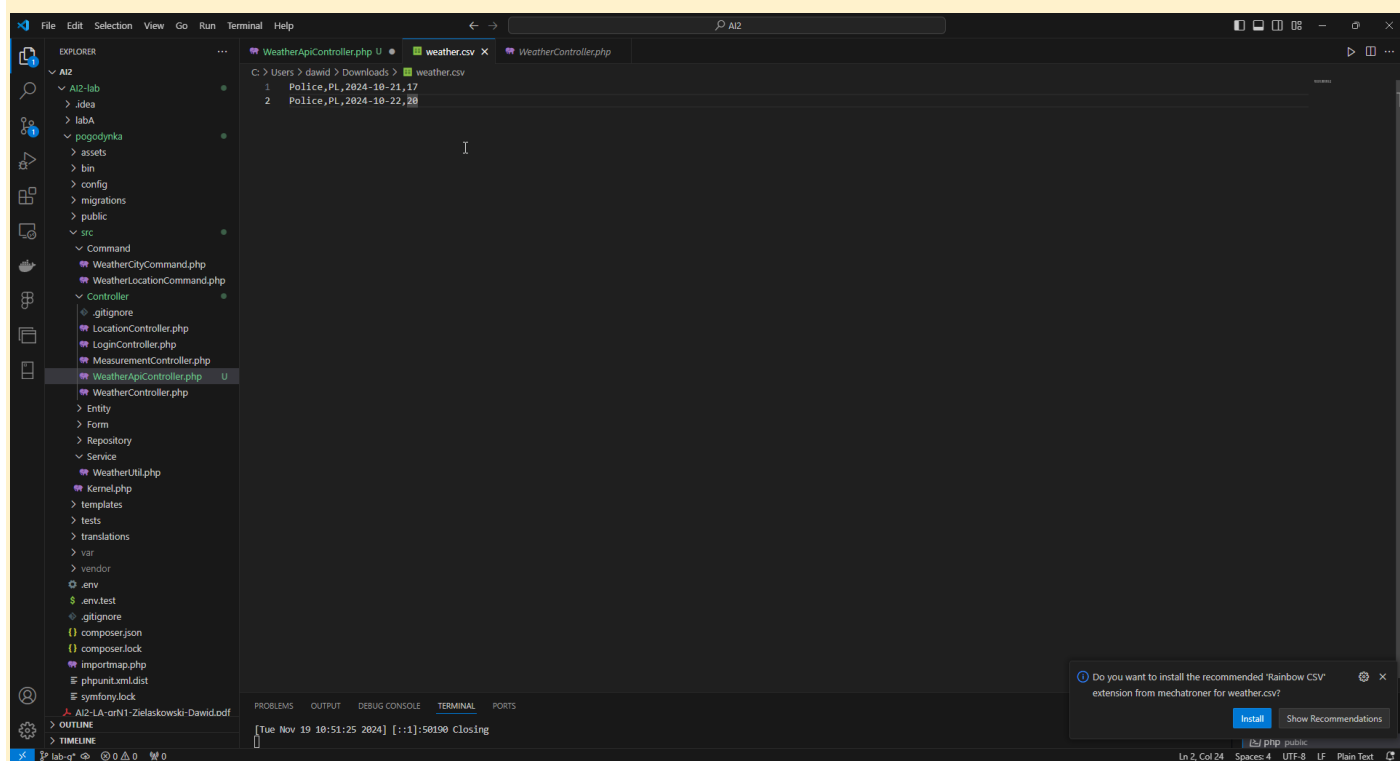


```
7 use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
8 use Symfony\Component\HttpFoundation\Response;
9 use Symfony\Component\HttpKernel\Attribute\MapQueryString;
10 use Symfony\Component\Routing\Annotation\Route;
11
12 class WeatherApiController extends AbstractController
13 {
14     #[Route('/api/v1/weather', name: 'app_weather_api', methods: ['GET'])]
15     public function index(
16         WeatherUtil $util,
17         #[MapQueryString('country')] string $country,
18         #[MapQueryString('city')] string $city,
19         #[MapQueryString('format')] string $format
20     ): Response
21     {
22         $measurements = $util->getWeatherForCountryAndCity($country, $city);
23
24         if ($format === 'csv') {
25             $csvData = array_map(fn(Measurement $m) => sprintf(
26                 '%s,%s,%s,%s',
27                 $city,
28                 $country,
29                 $m->getDate()->format('Y-m-d'),
30                 $m->getCelsius()
31             ), $measurements);
32
33             $response = implode("\n", $csvData);
34             return new Response($response, Response::HTTP_OK, [
35                 'Content-Type' => 'text/csv',
36                 'Content-Disposition' => 'attachment; filename="weather.csv"'
37             ]);
38         }
39         if ($format === 'json') {
40             return $this->json([
41                 'city' => $city,
42                 'country' => $country,
43                 'measurements' => array_map(fn(Measurement $m) => [
44                     'date' => $m->getDate()->format('Y-m-d'),
45                     'celsius' => $m->getCelsius(),
46                 ], $measurements),
47             ]);
48         }
49     }
50 }
```

Wstaw zrzut ekranu otrzymanego z kontrolera JSONa:



Wstaw zrzut ekranu otrzymanego z tego samego kontrolera CSV. **Uwaga! Poprawny CSV ma osobny wiersz dla każdego rekordu i NIE ma znaczników
. Jeśli otwierasz wynik w przeglądarce – podejrzaj źródła (Ctrl+U):**



Punkty:

0

1

WYKORZYSTANIE TWIG

W tej sekcji otrzymamy identyczne wyniki jak w poprzednich sekcjach, z wykorzystaniem szablonów TWIG do generowania odpowiedzi.

Utwórz pliki:

- templates/weather_api/index.csv.twig
- templates/weather_api/index.json.twig

W kontrolerze dodaj nowy opcjonalny parametr boolowski `twig`. Ustawienie jego wartości skutkować będzie renderowaniem odpowiedzi z wykorzystaniem TWIG:

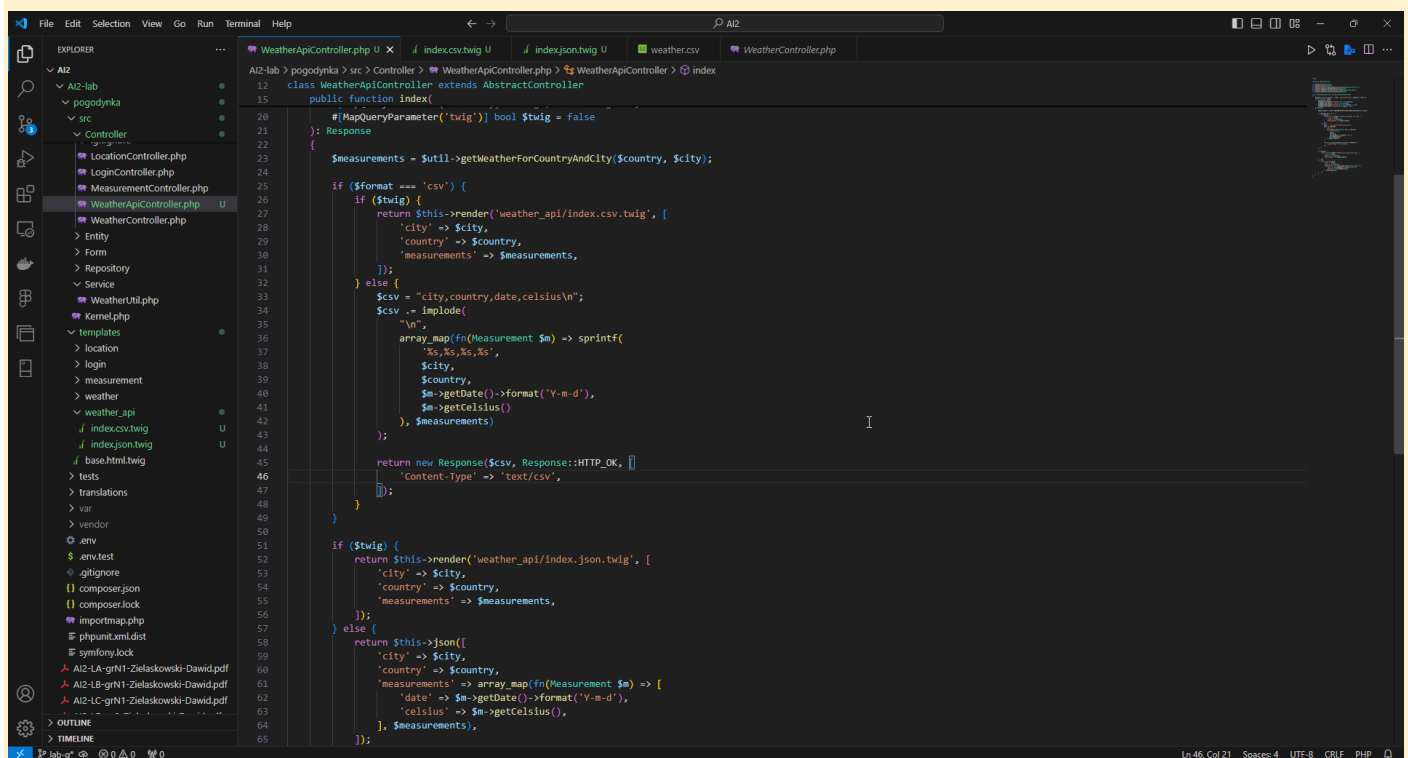
```
#[MapQueryParameter('twig')] bool $twig = false,
```

Przykładowy sposób wywołania generowania odpowiedzi z wykorzystaniem TWIG:

```
return $this->render('weather_api/index.csv.twig', [
    'city' => $city,
    'country' => $country,
    'measurements' => $measurements,
]);
```

Skopiuj teraz odpowiedzi CSV i JSON Twojego API w dotychczasowej wersji i wklej do szablonów TWIG. Następnie, wykorzystaj parametry `city`, `country` i `measurements` oraz instrukcje sterujące TWIG do zamiany tych statycznych odpowiedzi do postaci dynamicznej.

Wklej zrzut ekranu kodu kontrolera z obsługą przełączania formatu i twiga:



Punkty:	0	1
---------	---	---

Wklej zrzut ekranu kodu TWIG generowania odpowiedzi w formacie JSON. **UWAGA – renderuj odpowiedź „na piechotę”, a nie używając json_encode!**

```

1 {
2     "city": "Szczecin",
3     "country": "PL",
4     "measurements": [
5         {% for measurement in measurements %}
6         {
7             "date": "{{ measurement.date | date('Y-m-d') }}",
8             "celsius": "{{ measurement.celsius }}"
9         }{% if not loop.last %},{% endif %}
10        {% endfor %}
11    ]
12 }

```

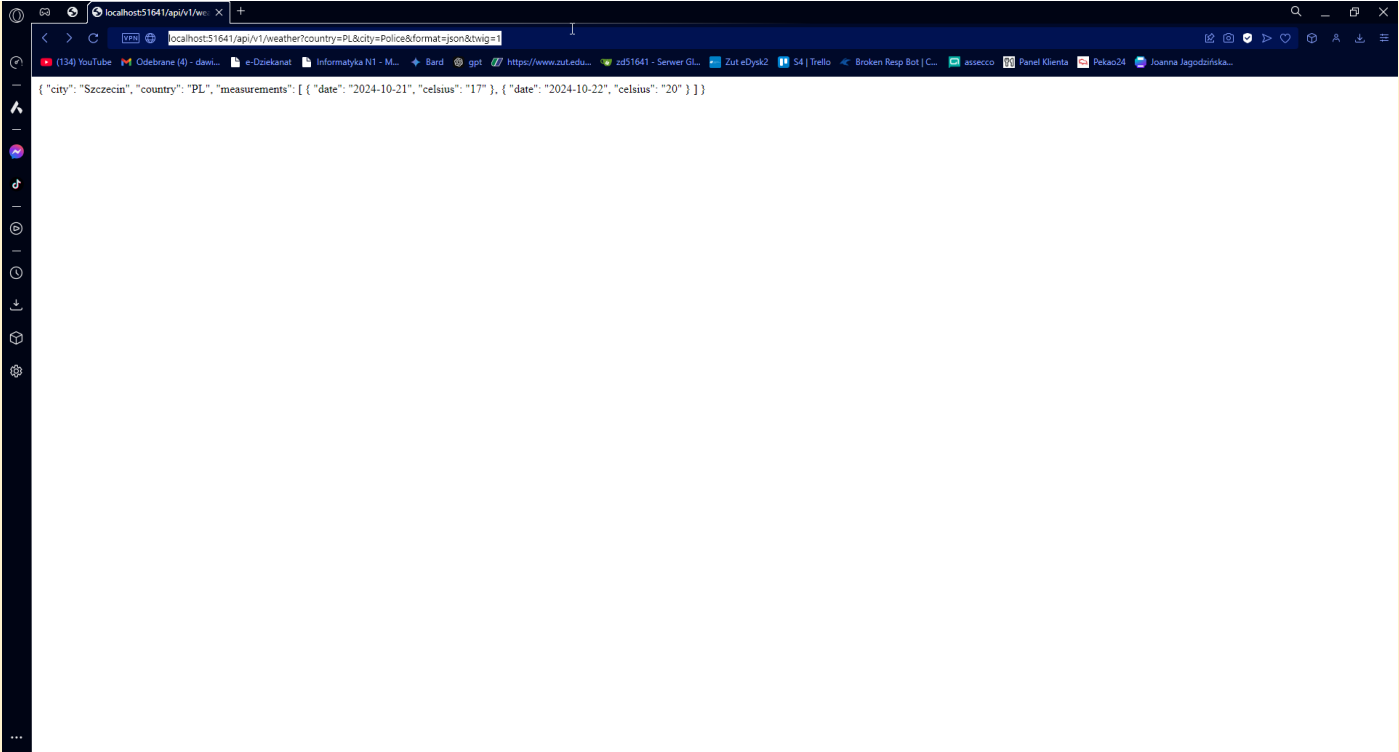
Terminal output:

```

PS C:\Users\dawid\Desktop\Studia\AI2> cd .\AI2-lab\
PS C:\Users\dawid\Desktop\Studia\AI2-lab> cd .\pogodynka\
PS C:\Users\dawid\Desktop\Studia\AI2-lab\pogodynka> cd .\public\
PS C:\Users\dawid\Desktop\Studia\AI2-lab\pogodynka\public> php -S localhost:51641
[Tue Nov 19 15:59:32 2024] PHP 8.2.4 Development Server (http://localhost:51641) started

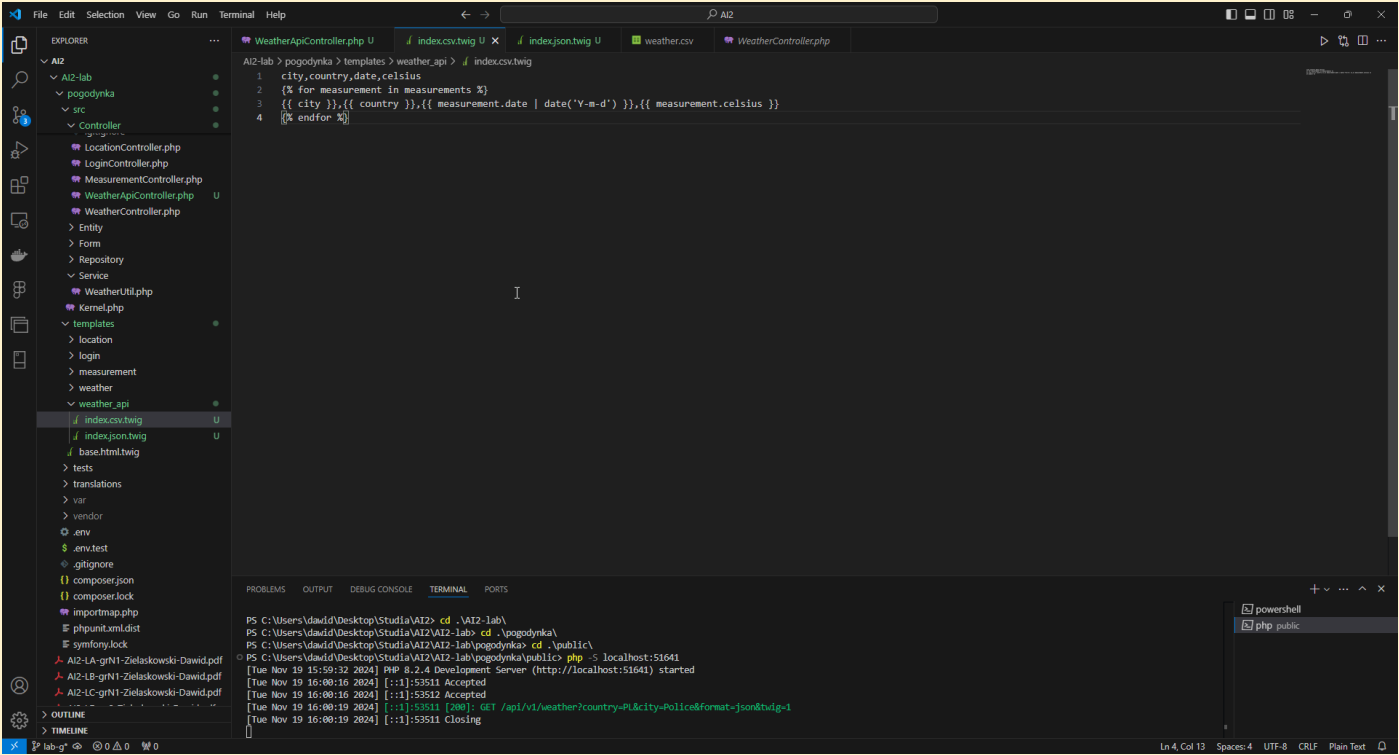
```

Wklej zrzut ekranu przykładowej odpowiedzi JSON wygenerowanej przez TWIG. Upewnij się, że zrzut ekranu zawiera całość adresu URL ze wszystkimi parametrami wywołania (&format=json&twig=1).

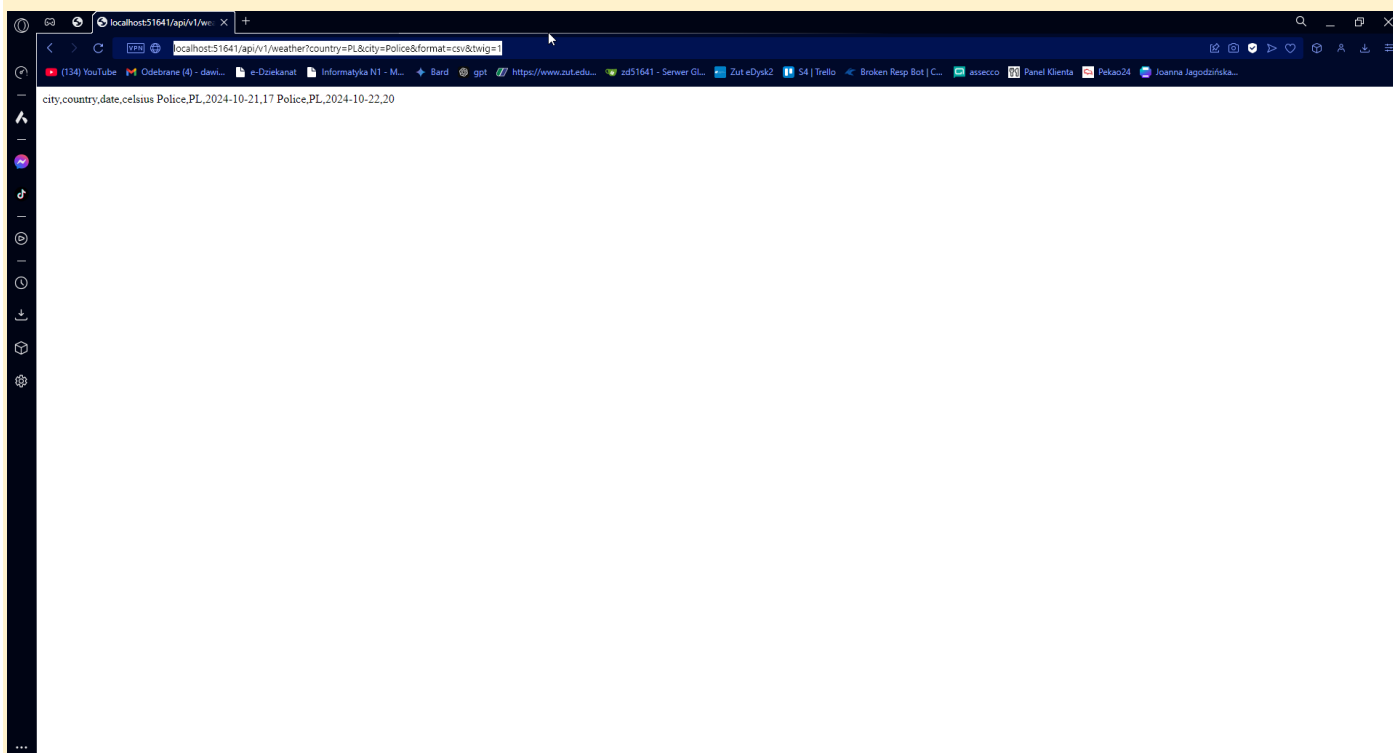


Punkty:	0	1
---------	---	---

Wklej zrzut ekranu kodu TWIG generowania odpowiedzi w formacie CSV. **Uwaga! KUDOS za renderowanie jednej linii odpowiedzi CSV w wielu liniach TWIGa – sprawdź funkcjonalność kontroli białych znaków w TWIG:**



Wklej zrzut ekranu przykładowej odpowiedzi CSV wygenerowanej przez TWIG. Upewnij się, że zrzut ekranu zawiera całość adresu URL ze wszystkimi parametrami wywołania (&format=json&twig=1).



Punkty:	0	1
---------	---	---

FAHRENHEIT

Do encji pomiarów dodaj metodę `getFahrenheit()`. Metoda ta powinna zwracać wartość `$this->getCelsius()` skonwertowaną do skali Fahrenheita. Formuła: $(0^{\circ}\text{C} \times 9/5) + 32 = 32^{\circ}\text{F}$

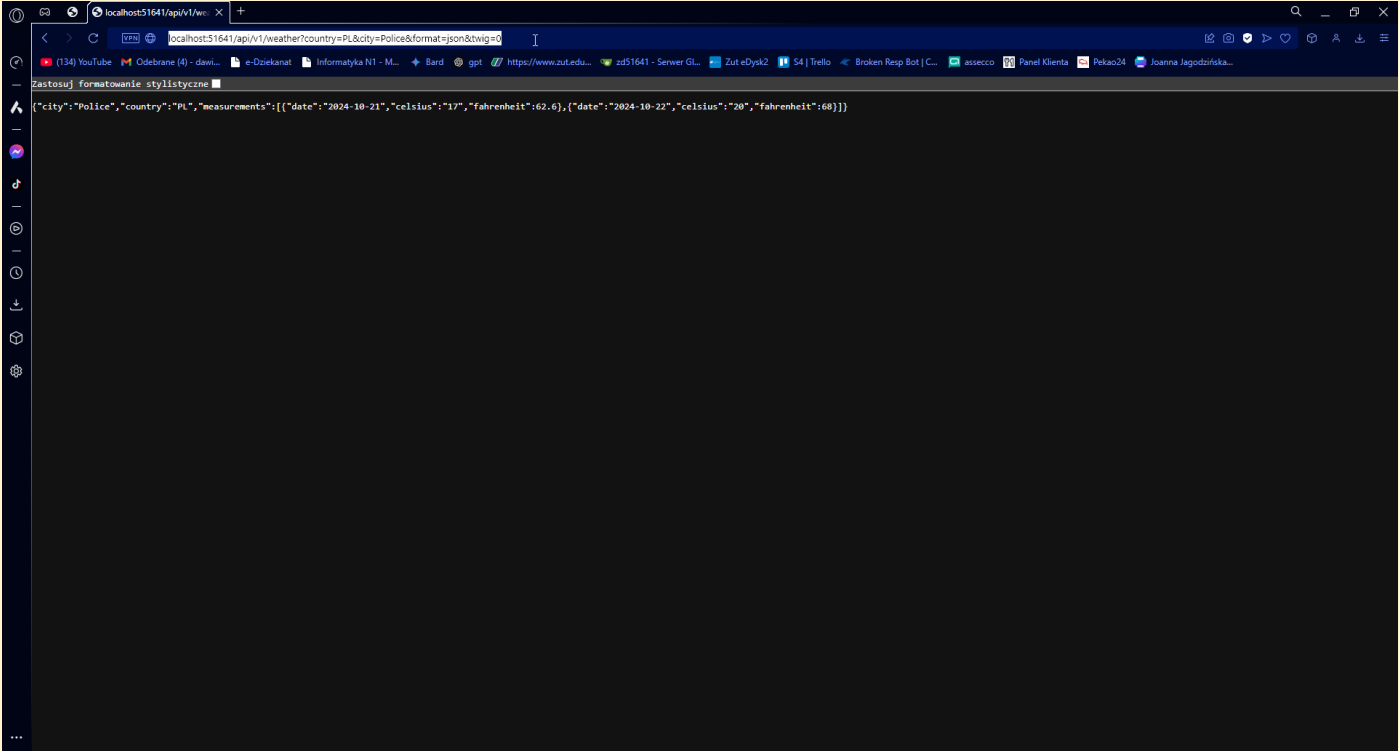
Zmodyfikuj wszystkie cztery odpowiedzi API (JSON, CSV), aby zwracały temperaturę w skali Celsjusza i Fahrenheita, przykładowo:

```
//...
    'date' => $m->getDate()->format('Y-m-d'),
    'celsius' => $m->getCelsius(),
    'fahrenheit' => $m->getFahrenheit(),
//...
```

Wklej zrzut ekranu kodu metody `getFahrenheit()`:

```
public function getFahrenheit(): ?string
{
    return ($this->celsius * 9 / 5) + 32;
}
```

Wklej zrzut ekranu przykładowej odpowiedzi JSON z uwzględnieniem obu skali temperatury:



Wklej zrzut ekranu przykładowej odpowiedzi CSV z uwzględnieniem obu skali temperatury:

```
C: > Users > dawid > Downloads > weather (5).csv
1 city,country,date,celsius,fahrenheit
2 Police,PL,2024-10-21,17,62.6
3 Police,PL,2024-10-22,20,68
```

Punkty:	0	1
---------	---	---

TEST JEDNOSTKOWY

Wykorzystaj metodę `make:test` do utworzenia szablonu testu jednostkowego:

```
php .\bin\console make:test

Which test type would you like?:
[TestCase      ] basic PHPUnit tests
[KernelTestCase] basic tests that have access to Symfony services
[WebTestCase   ] to run browser-like scenarios, but that don't execute JavaScript code
```

AI2 LAB G – Zielaskowski Dawid – Wersja 1

```
[ApiTestCase ] to run API-oriented scenarios
[PantherTestCase] to run e2e scenarios, using a real-browser or HTTP client and a real web server
> TestCase

Choose a class name for your test, like:
* UtilTest (to create tests/UtilTest.php)
* Service\UtilTest (to create tests/Service/UtilTest.php)
* \App\Tests\Service\UtilTest (to create tests/Service/UtilTest.php)

The name of the test class (e.g. BlogPostTest):
> Entity\MeasurementTest

created: tests/Entity/MeasurementTest.php

Success!

Next: Open your new test class and start customizing it.
Find the documentation at https://symfony.com/doc/current/testing.html#unit-tests
```

Zmień metodę `testSomething()` w utworzonym `tests/Entity/MeasurementTest.php` na `testGetFahrenheit`.

Zaimplementuj test, który utworzy nową encję pomiarów, a następnie kolejno:

- ustawi wartość stopni Celsjusza na 0 i sprawdzi czy `getFahrenheit()` zwraca poprawną wartość;
- ustawi wartość stopni Celsjusza na -100 i sprawdzi czy `getFahrenheit()` zwraca poprawną wartość;
- ustawi wartość stopni Celsjusza na 100 i sprawdzi czy `getFahrenheit()` zwraca poprawną wartość.

Pamiętaj, aby wartości stopni Celsjusza i Fahrenheita wewnątrz testu przekazywać jako wartości wpisane „na sztywno”, a nie wyliczane.

Pamiętaj, aby wartości stopni Celsjusza i Fahrenheita wewnątrz testu przekazywać jako wartości wpisane „na sztywno”, a nie wyliczane.

...maksymalnie dwa zdania wytłumaczenia dlaczego nie można użyć `getFahrenheit()` do wyliczenia wartości „w locie”...

Uruchom test z wykorzystaniem komendy:

```
php .\bin\phpunit
PHPUnit 9.6.13 by Sebastian Bergmann and contributors.

Testing
.
1 / 1 (100%)

Time: 00:00.230, Memory: 6.00 MB

OK (1 test, 3 assertions)
```

Wklej zrzuty ekranu całości kodu pliku `MeasurementTest.php`:

```

<?php

namespace App\Tests\Entity;
use App\Entity\Measurement;
use PHPUnit\Framework\TestCase;

class MeasurementTest extends TestCase
{
    public function testGetFahrenheit(): void
    {
        $measurement = new Measurement();

        // Test 1: 0 stopni Celsjusza
        $measurement->setCelsius(0);
        $this->assertEquals(
            32.0,
            $measurement->getFahrenheit(),
            '0°C = 32°F'
        );

        // Test 2: -100 stopni Celsjusza
        $measurement->setCelsius(-100);
        $this->assertEquals(
            -148.0,
            $measurement->getFahrenheit(),
            '-100°C = -148°F'
        );

        // Test 3: 100 stopni Celsjusza
        $measurement->setCelsius(100);
        $this->assertEquals(
            212.0,
            $measurement->getFahrenheit(),
            '100°C = 212°F'
        );
    }
}

```

Wklej zrzut ekranu wywołania i wyniku testów:

```

[ ] Find the documentation at https://symfony.com/doc/current/testing.html#unit-tests
● PS C:\Users\dawid\Desktop\Studia\AI2\AI2-lab\pogodynka> php .\bin\phpunit
PHPUnit 9.6.21 by Sebastian Bergmann and contributors.

Testing
.
1 / 1 (100%)

Time: 00:00.071, Memory: 6.00 MB

OK (1 test, 3 assertions)
○ PS C:\Users\dawid\Desktop\Studia\AI2\AI2-lab\pogodynka>

```

Upewnij się że wykonano 1 test i 3 asercje.

Punkty:	0	1
---------	---	---

DATAPROVIDER

W tej sekcji sprawdzimy więcej przypadków, również uwzględniających ułamki. Wykorzystamy `dataProvider`.
Utwórz funkcję `dataGetFahrenheit()`:

```
public function dataGetFahrenheit(): array
{
    return [
        ['0', 32],
        ['-100', -148],
        ['100', 212],
    ];
}
```

Nad testem dodaj adnotację:

```
@dataProvider dataGetFahrenheit
```

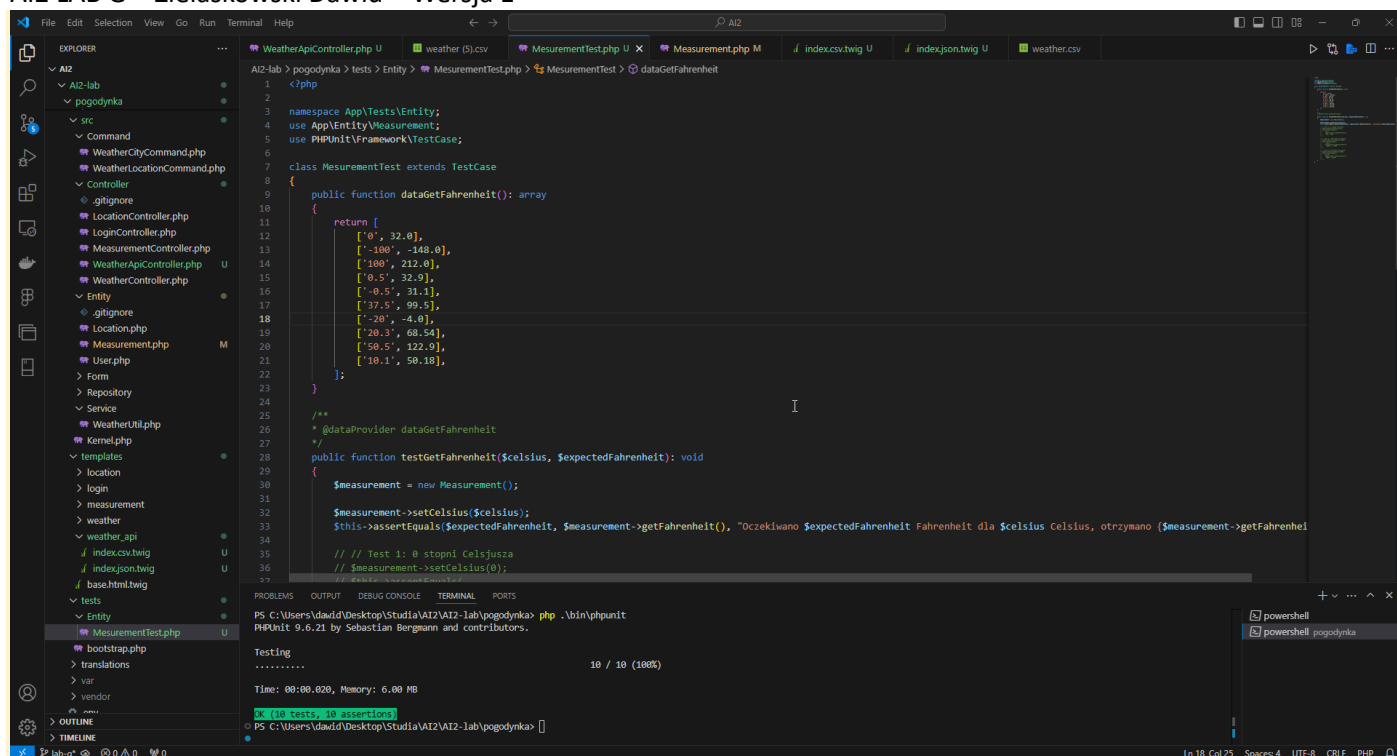
Zmień sygnaturę funkcji testu:

```
public function testGetFahrenheit($celsius, $expectedFahrenheit): void
```

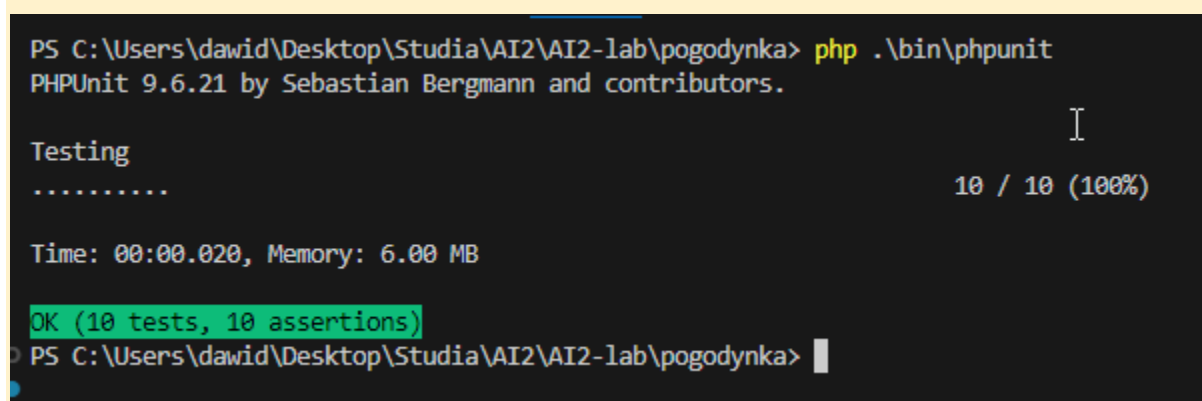
Zmodyfikuj kod funkcji w taki sposób, żeby zamiast „na sztywno” sprawdzać wartości 0, -100 i 100, wykorzystywał parametr `$celsius` i `$expectedFahrenheit`.

Uzupełnij dane wejściowe w `dataGetFahrenheit` do 10 wartości, również wykorzystujących ułamki, np. 0.5 stopnia Celsjusza to 32.9 stopnia Fahrenheita.

Wklej zrzuty ekranu całości kodu pliku `MeasurementTest.php`:



Wklej zrzut ekranu wywołania i wyniku testów:



Upewnij się że wykonano 10 testów i 10 asercji.

Punkty:	0	1
---------	---	---

COMMIT PROJEKTU DO GIT

Zacommituj zmiany. Wyślij zmiany do repozytorium (push). Upewnij się, czy wszystko dobrze się wysłało. Jeśli tak, to z poziomu przeglądarki utwórz branch o nazwie `lab-g` na podstawie głównej gałęzi kodu.

Podaj link do brancha `lab-g` w swoim repozytorium:

<https://github.com/dawid-zielaskowski/ai2-pogodynka/tree/lab-g>

...link, np. <https://github.com/ideaspot-pl/ai2-pogodynka-202310/tree/lab-g...>

PODSUMOWANIE

W kilku zdaniach podsumuj zdobyte podczas tego laboratorium umiejętności.

...podsumowanie...

Zweryfikuj kompletność sprawozdania. Utwórz PDF i wyślij w terminie.